Hoofdstuk 8

Semantische modellen

Business Intelligence is veel meer dan het bouwen van een datawarehouse. OLAP (Online Analytical Processing) of OLAP-kubussen zijn een traditionele component van BI. In het vorige hoofdstuk zijn ze al genoemd onder de noemer semantisch model. In dit hoofdstuk kun je lezen over OLAP, semantische modellen en kubussen. Wat is het en welke rol spelen ze?

8.1 Inleiding

Om goed, gemakkelijk en snel gegevens te kunnen analyseren zijn datamarts in de vorm van een sterschema uiteindelijk niet toereikend. Het sterschema is als het goed is intuïtief voor iemand die het gemodelleerde proces kent, voor de gebruiker dus. Hij herkent de dimensies als de grootheden waarmee hij dagelijks werkt en de feiten zijn de relevante getallen voor zijn rapporten. Waarom voldoet een datamart dan toch niet?

Laten we, om die vraag te beantwoorden, eerst eens kijken naar drie eisen die worden gesteld aan gespecialiseerde OLAP-databases.

- Een OLAP-database moet een conceptueel en intuïtief gegevensmodel opleveren.
- Een OLAP-database moet een hele goede performance hebben, ook bij grote datasets.
- Een OLAP-database moet (relatief) gemakkelijk voor analysedoeleinden nuttige berekeningen kunnen uitvoeren.

In de komende paragrafen komen deze aspecten nader aan bod.

8.1.1 Semantisch model

Hoewel zonet is gezegd dat een sterschema – als het goed is – intuïtief is, blijkt dat in de praktijk toch tegen te vallen. Een stermodel heeft de volgende tekortkomingen om het een intuïtieve bron voor eindgebruikers te laten zijn:

- Een stermodel is een technisch gegevensmodel met 'technische' kolommen als ProductKey en SourceKey.
- Een stermodel bevat voor eindgebruikers irrelevante technische kolommen om Slowly Changing Dimensions bij te houden zoals RowStartDate en RowEndDate.
- Een stermodel bevat auditing-gerelateerde kolommen.
- Kolomnamen sluiten niet aan bij het bedrijfsjargon.
- De tabelstructuur levert problemen op voor query-generatoren.
- Nuttige berekende kolommen en meetwaarden ontbreken.

Niet-relevante kolommen

Een stermodel is nodeloos complex omdat er voor de analysedoeleinden allemaal overbodige technische kolommen in zitten. Deze complexiteit kan de acceptatie van de eindgebruikers in de weg staan.

Een semantisch model bevat alleen de voor de gebruikers relevante kolommen en entiteiten.

Kolomnamen
Naast kolommen die voornamelijk technisch van aard zijn, hebben kolommen vaak ook niet de goede namen. Stel bijvoorbeeld dat je van producten het gewicht in aantal gram bijhoudt. Je hebt dan waarschijnlijk een kolom met een naam als GewichtInGram. In relationele databases is het een best practice geen spaties in namen te gebruiken. Spaties kunnen allerlei problemen opleveren (overigens voornamelijk als er slordig wordt geprogrammeerd). Daarom is besloten van de eigenlijke drie woorden er één te maken en met hoofdletters de oorspronkelijke woorden toch nog te benadrukken. Vanuit een databaseperspectief is dit zeker aanvaardbaar, maar niet vanuit een rapportageperspectief.

Rapporten moeten heldere, leesbare kolomnamen hebben, die ook nog eens consistent over alle rapportages heen gebruikt worden. Deze namen komen overeen met de taal van het bedrijf. Dat kan een spreektaal zijn, of het jargon van het specifieke bedrijf. Met namen als GewichtInGram moet de maker van rapporten de naam op het rapport gaan wijzigen. Iedereen zal dat op zijn eigen manier doen, wat de eenduidigheid niet ten goede zal komen.

■ Headers op rapporten moeten duidelijk zijn, in de taal van het bedrijf geschreven eenduidig over alle rapporten heen.

Erger zou nog zijn geweest als de kolom Gewicht had geheten. Het is nu een kwestie van tijd dat er rapporten komen met kolommen *Gewicht, Gewicht in gram, Gewicht in KG*. Drie namen voor hetzelfde databaseveld met ook nog eens een verkeerde interpretatie.

In een semantisch model worden kolomnamen gebruikt zoals later de kopteksten van kolommen op rapporten moeten zijn. Namen hoeven op rapporten niet meer te worden aangepast. De eenduidigheid van de rapportages wordt beter.

De tabelstructuur levert problemen op voor query-generatoren
Een sterschema bestaat uit meer tabellen: de feitentabel met daaraan via foreign keys gelinkte dimensietabellen. Iemand die rapporten of analyses uitvoert met het sterschema als basis, moet wel op de hoogte zijn van hoe deze relaties lopen.

Er zijn veel grafische query-tools op de markt die gebruikers in staat stellen met drag-and-drop query's te schrijven. Vaak gaat dat automatisch genereren van query s goed. Maar als er meer dan één relatie is gedefinieerd tussen twee tabellen, wordt waarschijnlijk de verkeerde query gegenereerd. Ook als er geen relaties zijn gedefinieerd gaat het nogal eens fout. In die gevallen zal de gebruiker zelf inzicht moeten hebben in het technische aspect van de database.

Informatie die laat zien wat echt is gebeurd, is vaak redelijk gemakkelijk te tonen. Bijvoorbeeld een lijst van alle landen met daarachter de in dat land gerealiseerde omzet in een bepaalde periode. Maar stel nu eens dat er landen zijn waar in de gekozen periode geen omzet was? Kun je ook een lijst maken van alle landen die in die periode geen omzet hadden? Dat kan een zeer relevante vraag zijn. In technische termen is dat het verschil tussen een inner en een outer join. Gebruikers hebben dus technische kennis van SQL nodig.

In een semantisch model wordt vastgelegd hoe verschillende entiteiten uit de organisatie verband met elkaar houden. Deze informatie stelt query-tools in staat altijd de juiste query's te schrijven zonder dat de gebruiker kennis van de onderliggende structuur of oplossing nodig heeft. Kennis van de bedrijfsprocessen is alles wat de gebruiker nodig heeft.

Berekende kolommen

Query-tools stellen gebruikers in staat bestaande kolommen te selecteren. Soms moeten kolommen echter nog berekend worden. Om een simpel voorbeeld te noemen: in de database staat een prijs exclusief btw en op het rapport wil je de prijs inclusief btw tonen. Dit is natuurlijk een makkelijke berekening die in elke grafische tool ook gemakkelijk zal zijn. Maar de berekeningen kunnen veel complexer worden en het gebruik van moeilijke functies noodzakelijk maken. Nu moet een gebruiker opeens weten wat functies zijn en hoe ze te gebruiken.

In een goed semantisch model zal bovenstaande niet nodig zijn. Met een goed model bedoelen we een model dat is gebaseerd op een gedegen informatieanalyse. Vooraf is (voor zover mogelijk) duidelijk geworden welke velden gebruikt gaan worden. In het technisch ontwerp is bepaald waar die velden gedefinieerd worden. Dat kan zijn als kolommen in de datamart, het kunnen ook berekende velden in het semantische model zijn. De complexe berekening met functies zitten nu in het model, de gebruiker kan weer 'gewoon' de benodigde kolom selecteren.

Een **semantisch model** vertaalt een technische database-implementatie in een voor eindgebruikers leesbaar en begrijpelijk model dat bruikbaar is zonder over (veel) technische kennis te beschikken.

Belangrijker nog dan berekende kolommen zijn berekende meetwaarden. In veel rapportages zal een gebruiker bijvoorbeeld de omzet van een bepaalde periode willen vergelijken met de omzet in diezelfde periode een jaar geleden. Met periode kan hier weken, maanden, kwartalen of nog iets anders bedoeld worden. In een relationele datamart is dit lastig vast te leggen. Bij het gebruik van SQL-query's heeft de rapportmaker SQL kennis nodig, is er kans op fouten en zal de performance in het geding komen. In een goede OLAP-database kunnen dit soort berekende waarden (relatief) gemakkelijk gedefinieerd worden. De rapportmaker hoeft nu enkel de al voor gedefinieerde berekende waarde te gebruiken.

Tot slot

We begonnen deze paragraaf met te stellen dat een OLAP-database een conceptueel en intuïtief gegevensmodel moet opleveren. Dat wordt bereikt door die database een semantische model te laten zijn.

Twee belangrijke criteria bepalen hoeveel meerwaarde een semantisch model heeft:

- Het technische niveau van diegenen die rapporten gaan bouwen en die analyses gaan doen;
- Hoe goed de gebruikte datamart aansluit bij het gebruik.

Als alleen BI-specialisten rapporten maken en analyses doen, dan mag worden aangenomen dat er geen technische beperkingen zijn bij deze mensen. Ze zullen altijd in staat zijn de juiste query's te schrijven.

Merk op dat het bouwen van een semantisch model overeenkomt met een top-downbenadering.

8.1.2 Performance

De tweede eis die we aan het begin van deze paragraaf hebben gesteld aan OLAP-databases is een goede performance. In dit boek hebben we het al veel gehad over performance. In eerdere hoofdstukken is benadrukt dat de acceptatie van de BI-oplossing door de eindgebruikers een kritische succesfactor is voor de oplossing. Als gebruikers het systeem (te) traag vinden, gaat het niet goedkomen met die acceptatie en dus niet met de BI-oplossing. Bij het gebruik van datamarts komt de performance op twee vlakken in het geding.

- Ten eerste is er de tijd die het duurt om de juiste query te schrijven.
- Ten tweede is er de tijd die het duurt om de query uit te voeren.

Een OLAP-database zou beide zorgen uit de weg moeten nemen.

Benodigde tijd om query te schrijven

Voorbeeld
Iemand zit met een vraag. Hij belt met de Business Intelligence-afdeling met het verzoek of zij een query voor hem kunnen schrijven. Dat kan bijvoorbeeld zijn omdat deze gebruiker niet de technische kennis heeft om de benodigde informatie zelf uit het systeem te halen. Dat kan ook zijn omdat de gebruiker niet de juiste rechten heeft om zelf de gegevens op te vragen. Of misschien omdat hij zelf geen zin heeft om achter de benodigde informatie aan te gaan. Uiteraard kan de BI-afdeling aan de gestelde vraag voldoen. Vandaag zijn ze echter net bezig met een migratie naar een nieuwere versie van het DBMS, maar overmorgen zullen ze zeker de query gaan schrijven. In de communicatie ging iets fout waardoor initieel de verkeerde query is geschreven. Een dag later wordt dat rechtgezet.

Er is nu drie en een halve dag verstreken tussen het ontstaan van de vraag en het krijgen van het antwoord. Met een beetje pech is de informatie al niet meer relevant.

Met een goed semantisch model zou een dergelijke situatie niet meer voor moeten kunnen komen. Belangrijk is weer de acceptatie. Die moet ervoor zorgen dat de derde reden waarom de gebruiker aan het BI-team vraagt om informatie (hij heeft geen zin om het zelf te doen) niet meer voorkomt. Het model moet dusdanig intuïtief zijn dat de gebruiker geen technische tekortkomingen tegenkomt. Ten slotte moet het model een security-implementatie bevatten zodanig dat de gebruiker alle voor hem relevante gegevens kan opvragen.

Query-performance

Nadat de benodigde query is geschreven is er natuurlijk nog de performance van de query zelf. Hoe lang duurt het om de query uit te voeren? In hoofdstuk 5 is een aantal 'trucs' besproken die kunnen helpen om de performance van een database positief te beïnvloeden. Voor het merendeel gingen deze trucs ervan uit dat precies bekend was welke query's werden gedraaid, om vervolgens die specifieke query's sneller te maken. Voor veelgebruikte rapporten is dat geen vreemde aanname. In een analyseomgeving gaat dat echter meestal niet op. Er kunnen willekeurige vragen gesteld worden en er zullen telkens weer andere vragen opduiken. Veelal zullen analyses beginnen op hoog aggregatieniveau. Dus grote datasets geaggregeerd tot een aantal getallen. In technische termen: dure group by's. In vervolgstappen is het niet onwaarschijnlijk dat er drill-downs uitgevoerd worden om informatie op lagere aggregatieniveaus te bekijken.

Met betrekking tot query-performance gaat er een verschil optreden tussen producten die 'alleen' een semantisch model zijn, echte OLAP-databases, en de veel nieuwere in-memory producten. Er zijn veel producten op de markt die semantische modellen leveren. Bekende voorbeelden zijn de Universe van Business Objects en het framework van Cognos. Deze modellen moet je eigenlijk zien als SQL-query-generatoren. Ze bieden de gegevens via een intuïtief model aan de gebruikers aan. De gebruiker sleept een rapport bij elkaar en het model genereert de onderliggende SQL-query die wordt afgevuurd op de datamart.

Bij deze tools is performance juist vaak een probleem. De automatisch gegenereerde query's zijn bijna nooit heel efficiënt. En zelfs als ze dat wel zijn, is de beste performance die je kunt krijgen de performance die het DBMS te bieden heeft. Met zware aggregaties op grote, gejoinde tabellen, kan dat wel eens een probleem zijn. Uiteraard is dat een kwestie van hoe groot de database is, welk DBMS je gebruikt en welke query's je uitvoert.

Een OLAP-database moet een betere performance bieden door de gegevens zelf in een ander formaat op te slaan. De gegevens komen dan niet meer uit de (relationele) datamart maar rechtstreeks uit de OLAP-database. In paragraaf 8.2 kun je daar meer over lezen.

De laatste tijd komen er steeds meer in-memoryproducten. Met de huidige hardware kunnen grote datasets in het werkgeheugen van servers geladen worden. Klassieke databasesystemen en ook OLAP-databases gaan uit van de opslag van de gegevens op disk. Schijven zijn echter eindeloos veel trager dan geheugen. Door gegevens permanent in het geheugen te houden kan de bottleneck die vaak gevormd wordt door de IO (verplaatsen van gegevens van schijf naar geheugen en vice versa) geëlimineerd worden. Bovendien kunnen andere structuren gebruikt worden om nog eens extra voordeel te creëren. Voorbeelden van in-memoryproducten zijn SAP Hana, QlikView en het op PowerPivot gebaseerde Tabular Model van Microsoft SQL Server. In paragraaf 8.3 kun je daar een voorbeeld van zien.

Met een OLAP-database of een in-memorydatabase kunnen performanceproblemen die bij datamarts een rol spelen voorkomen worden.

8.1.3 Query-performance en Big Data

Op dit punt is het goed om even terug te keren naar Big Data. Als de query-performance slecht is vanwege de grootte van de gebruikte datasets, moeten we dan geen Big Data-clusters gebruiken? Hoewel dat aannemelijk klinkt, is het antwoord nee.

Het doel van OLAP-databases is dat mensen interactief, realtime, 'spelen' met gegevens. Daar hoort bij dat je ad hoc andere dwarsdoorsnedes wil maken, drill-downs of drill-throughs wilt doen, of andere meetwaardes wilt bekijken. Dat is niet de sterkte kant van Big Data-tools. Het is wel de kracht van kubussen.

Veel Big Data-architecturen verdelen de gebruikte technieken in drie lagen. Dit zijn de:

- batch layer;
- speed layer;
- serving layer.

Batch layer

De meeste Big Data-tools werken nog met jobs. Jobs zijn stukken werk die zich goed laten schedulen. De doorlooptijd van deze jobs is vaak zodanig dat je er niet op wilt wachten. Bij kleine datasets praten we vaak al over minuten, bij grote datasets kan dat uren duren. Het is het handigst deze jobs in de nacht te laten uitvoeren als niemand al duimendraaiend zit te wachten. We noemen dat batchverwerking. In een Big Data-architectuur vormen deze jobs de batch layer.

Speed layer

Om realtimeanalytics mogelijk te maken, kunnen we naast de batch layer ook een zogenaamde speed layer bouwen. Deze gebruikt vaak streamingtools zoals Microsoft Azure Stream Insight in combinatie met Event Hub of IoT hub. Met deze tools kunnen we gegevens in realtime analyseren en de resultaten gebruiken.

Serving layer

De serving layer dient om de gegevens aan te bieden aan de gebruikers. Hij kan de resultaten van de batch en de speed layer combineren, maar dat is niet noodzakelijk. Een OLAP-database zoals een kubus kan heel goed fungeren als de serving layer. Sterker nog, dat is een van de belangrijkste doelen van een OLAP-database.

Met andere woorden: de batch layer zal de heavy lifting van veel Big Data analyse en preparatie doen. Via de serving layer, ofwel via een OLAP-database of semantisch model, worden de resultaten beschikbaar gesteld aan de gebruikers.

8.1.4 OLAP query engine

De derde eis die we aan het begin van deze paragraaf hebben gesteld aan OLAP-databases is de capaciteit om berekende velden te maken. De mogelijkheden die je hier vindt, hangen sterk af van het product waar je mee werkt. Goede OLAP-databases hebben krachtige scripttalen om berekeningen te doen.

8.1.5 Tot slot

Hoewel sterschema's voor technici een intuïtieve omgeving vormen die de processen binnen de organisatie beschrijven, laten ze voor businessanalisten en rapportbouwers nog wel eens te wensen over. Semantische modellen kunnen helpen de problemen op te lossen. Modellen laten de beschikbare gegevens zien aan de gebruikers in hun eigen taal zonder technische randzaken. Met de juiste tools is nu alleen nog maar inhoudelijke kennis nodig om met de gegevens te werken. Technische kennis is niet meer nodig, die is al verwerkt in het model.

Ook de performance van sterschema's kan problemen opleveren. Producten die alleen een model zijn, blijven last houden van dat probleem. OLAP-databases (OLAP-kubussen) kunnen dat probleem oplossen. Naast kubussen zijn er steeds meer producten op de markt die de performance aanpakken door volledig vanuit het werkgeheugen van de server te werken, de in-memorymodellen of databases. In paragraaf 8.2 kun je meer lezen over kubussen en in paragraaf 8.3 werken we een voorbeeld uit van een in-memorymodel.

8.2 OLAP-kubus

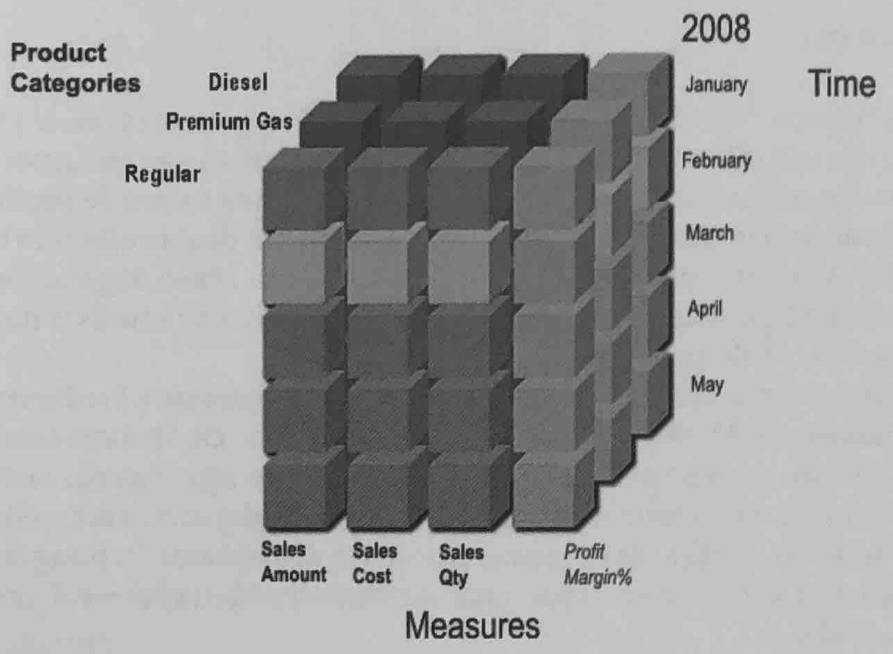
Zoals je in de vorige paragraaf hebt kunnen lezen is een OLAP-database een oplossing om eindgebruikers met goede performance zelf analyses te laten uitvoeren. De eindgebruikers hebben weinig technische kennis nodig om dit te kunnen.

De analyses waar het hier om draait, zijn veelal multidimensionele analyses, waarbij vanuit verschillende invalshoeken (dimensies) wordt gekeken naar meetwaarden zoals omzet en kosten. Dit wordt ook wel '*to slice and dice*' genoemd. Willekeurig, telkens met een ander perspectief, door de gegevens browsen. Free form-analyses kunnen uitvoeren zonder vast te zitten in vooraf vastgestelde vragen. Out of the box kunnen denken en analyseren. Dat is wat analisten willen en dat is wat kubussen beloven. Maar wat is nu eigenlijk een OLAP-database en een kubus?

8.2.1 Wat is een kubus?

Over het algemeen zijn OLAP-database en OLAP-kubus, of kortweg kubus, synoniemen. Een derde naam is multidimensionele database. Deze naam benadrukt dat het gaat om een analysedatabase waarbij vanuit meerdere perspectieven naar de gegevens gekeken moet kunnen worden.

De term kubus is zowel heel goed als heel slecht gekozen. Goed, omdat het in eenvoudige omgevingen ruimtelijk inzichtelijk kan maken wat (ongeveer) bedoeld wordt met een multidimensionele database. Slecht, omdat een kubus een driedimensionaal object is wat lang geen recht doet aan de complexiteit van de multidimensionele database die je wilt maken.



Figuur 8.1 *OLAP-kubus*

Figuur 8.1 laat zien wat een kubus is. Je ziet een kubus met twee dimensies en vier feiten. Een kubus is opgebouwd uit allemaal kleine kubussen. De kleinste kubus is een cel, een blokje dat één waarde kan bevatten. Op een van de assen van de kubus staan alle feiten. Op de andere assen staan de dimensies. Omdat het plaatje een driedimensionaal object is, blijven er twee assen over voor de dimensies. In figuur 8.1 zijn dat de dimensies Productcategorie (Product Category) en Tijd (Time). In de praktijk kan een kubus vele dimensies hebben.

Op elke as van de kubus staan de zogenaamde members. Members zijn gegevens in de database. Blijkbaar is er in het voorbeeld sprake van productcategorieën Diesel, Premium Gas en Regular. Dat is de inhoud van de productdimensie in het datawarehouse.

Members in een kubus worden gevormd door de inhoud van een kolom in een datawarehouse.

Bij het bepalen van de hoeveelheid assen die een kubus heeft, moet je onderscheid maken tussen de assen en de dimensies. In hoofdstuk 4 over dimensioneel modelleren heb je geleerd dat er een vuistregel is die zegt dat een stermodel uit niet meer dan zeven dimensies moet bestaan. Dat vertaalt zich echter niet naar een kubus met zeven assen. Het zal een veelvoud daarvan zijn.

Stel dat je naast de naam van een product ook het merk van het product bijhoudt. Op de kubus zou dit tot een aparte as leiden die bevolkt wordt met alle verschillende merken die in de database voorkomen. Een kolom prijs leidt tot een as die bevolkt wordt met alle prijzen vanuit de database. Eigenlijk geldt dat voor elk attribuut van elke dimensie een as wordt gemaakt die bevolkt wordt door een SELECT DISTINCT op de bijbehorende kolom in de dimensietabel. Met zeven dimensies die elk bestaan uit

twintig kolommen, hebben we het dus al over honderdveertig assen. Probeer dat maar eens in een plaatje te tekenen.

8.2.2 Waarom kubussen?

De mooiste beschrijving van wat een kubus is, is de volgende: *Excel on steroïds*! Wat is nou een kubus en waarom zijn ze zo handig?

Jaren geleden deed Microsoft nog helemaal niets op het gebied van Business Intelligence maar ze zagen wel aankomen dat het een grote markt ging worden die ze niet links konden laten liggen. Ze hebben toen laten uitzoeken wie de grote spelers op de BI-markt waren en welke producten die spelers maakten. Verrassenderwijs kwam daaruit naar voren dat Microsoft zelf marktleider was op het gebied van BI-software. En wel met Excel!

Business Intelligence was heel lang, en is bij sommige bedrijven nog steeds: export maken van gegevens uit een database en die gegevens analyseren met behulp van Excel. Tegenwoordig heeft Microsoft die boodschap begrepen en Excel wordt op de markt gezet als belangrijke Business Intelligence front-endtool. Ook veel nieuwe features van Excel zijn gericht op BI.

De vraag rijst waarom Excel, zelfs toen Microsoft het nog niet wist, zo'n belangrijke BI-tool was. Daar zijn, naast de populariteit van Office en het wijdverbreid zijn van Excel, twee hoofdredenen voor aan te wijzen.

- de draaitabel;
- formules en functies.

Draaitabel

Een draaitabel (matrix, pivottable) is een handige en veelgebruikte manier om grote hoeveelheden gegevens eenvoudig inzichtelijk te maken. Zet de namen van alle verkopers op de verticale as, de maanden van het huidige jaar op de horizontale as en je hebt snel inzichtelijk welke verkoper het in welke maanden goed of slecht heeft gedaan. Potentieel liggen hier honderden, zo niet duizenden, records aan ten grondslag, toch is het inzicht snel tot stand gekomen.

In het boek *Business Intelligence – Making better decisions faster* (Vitt, Luckevich & Misner, 2002) staat het volgende voorbeeld. Op een gewone database worden drie query's afgevuurd die respectievelijk de omzet per kwartaal, per markt en per product laten zien. Onderstaand zie je het resultaat:

Periode	Omzet
Q1	€ 16.000
Q2	€ 16.000
Totaal	€ 32.000

Plaats	Omzet
Amsterdam	€ 8.000
Rotterdam	€ 8.000
Den Haag	€ 8.000
Utrecht	€ 8.000
Totaal	€ 32.000

Product	Omzet
Appels	€ 8.000
Kersen	€ 8.000
Druiven	€ 8.000
Meloen	€ 8.000
Totaal	€ 32.000

Figuur 8.2 *Eendimensionale query's*

Dergelijke gegevens zijn redelijk saai en leiden niet tot speciale inzichten. Onduidelijk is welke vraag we verder nog kunnen stellen. Totdat we bedenken dat we geen eendimensionale vragen moeten stellen, maar multidimensioneel moeten denken en het resultaat moeten weergeven in de vorm van een draaitabel. Die gegevens kunnen er ook uitzien zoals in figuur 8.3.

		Amsterdam	Rotterdam	Den Haag	Utrecht	Totaal
Q1	Appels	-	-	€ 2.500	€ 1.500	€ 4.000
	Kersen	-	-	€ 2.000	€ 2.000	€ 4.000
	Druiven	€ 1.000	€ 3.000	-	-	€ 4.000
	Meloen	€ 2.000	€ 2.000	-	-	€ 4.000
	Totaal	€ 3.000	€ 5.000	€ 4.500	€ 3.500	€ 16.000
Q2	Appels	€ 4.000	-	-	-	€ 4.000
	Kersen	€ 1.000	€ 3.000	-	-	€ 4.000
	Druiven	-	-	€ 1.500	€ 2.500	€ 4.000
	Meloen	-	-	€ 2.000	€ 2.000	€ 4.000
	Totaal	€ 5.000	€ 3.000	€ 3.500	€ 4.500	€ 16.000
Totaal		€ 8.000	€ 8.000	€ 8.000	€ 8.000	€ 32.000

Figuur 8.3 *Multidimensionele draaitabel*

Als je de gegevens die deze draaitabel bevat als gewone rijen zou weergeven, dan zijn dat minimaal vijftien rijen. Daar zitten dan de totalen niet bij. Bovendien zitten daar de records niet bij die in de draaitabellen de lege cellen representeren. En juist die lege cellen geven hier verrassende en belangrijke informatie. Hoewel alle steden en alle producten het even goed lijken te doen (bij de eendimensionale views staat de omzet overal op dezelfde achtduizend), blijken sommige producten in sommige steden helemaal niet verkocht te worden!

In de praktijk zijn de in de draaitabellen getoonde getallen aggregaties van vele rijen die corresponderen met individuele transacties. Uit een lijst van honderden rijen had je nooit het inzicht gehaald dat de draaitabel wel heeft gegeven.

Dit voorbeeld laat zien wat de kracht van draaitabellen is. Met Excel is het gemakkelijk om draaitabellen te maken. De kracht van het resultaat en de eenvoud van het maken verklaren het succes op dit gebied van Excel.

De stap naar een kubus is nu niet meer zo groot. Een draaitabel is eigenlijk een kleine kubus met maar twee assen. De werkelijkheid is complexer dan de getoonde draaitabel. We hebben niet alleen omzet, maar ook het aantal verkochte stuks fruit. Er moet dus een cel bij voor dit nieuwe feit. Kwartaal kan verder uitgesplitst worden naar maanden. Daar moet een as voor komen. Er is nog een dimensie verkoper waar een as voor moet worden gemaakt. In essentie blijft het hetzelfde, een draaitabel met op de assen members van dimensies (Appels, Kersen, Amsterdam, Utrecht, ...) en op de snijpunten (de cellen) de feiten. Dat is precies wat een kubus is. Een kubus is dus een draaitabel, maar dan in het groot: *on steroïds*!

Functies en formules - MDX

Een tweede reden van het succes van Excel zijn de functies en formules. Die stellen de gebruiker relatief eenvoudig in staat bewerkingen toe te passen op de gegevens. De tegenhanger daarvan in kubussen is MDX, Multi Dimensional eXpressions. Van MDX kan zeker gezegd worden dat het krachtig is, helaas wordt het door de meeste mensen niet ervaren als makkelijk, in tegendeel zelfs.

MDX moet gezien worden als de tegenhanger van SQL voor kubussen. Wat SQL is voor relationele databases, is MDX voor kubussen. MDX is de query-taal van kubussen. Net zoals je SQL gebruikt om gegevens uit een relationele database te halen, gebruik je MDX-query's om gegevens uit een kubus te halen. Helaas is de syntax van beide talen wel anders. Dat kan ook niet anders. Een multidimensionele database is totaal anders van opzet dan een relationele database en dat heeft zijn weerslag op de bijbehorende taal.

Multi Dimensional eXpressions (MDX) is de taal die hoort bij OLAP-kubussen.

Zoals in paragraaf 8.1.3 is gezegd, heeft een OLAP-database een krachtige query engine nodig om met multidimensionele analyses om te kunnen gaan. Denk bijvoorbeeld eens aan een berekening van marge. Je hebt een kolom voor kosten en een kolom voor omzet. De marge in procenten is:

((omzet – kosten) / omzet) * 100%

In bijna alle programmeertalen is dit een eenvoudig te schrijven formule. Maar stel dat je in een SQL-query deze formule toepast en in de draaitabel van figuur 8.3 de omzet vervangt door marge. De resultaten zullen totaal verkeerd uitpakken!

Als de marge die je in Amsterdam haalt 10% bedraagt en de marge in Utrecht is 20%, dan is uiteraard de totale marge niet 10+20 is 30%. Was het maar waar! Je kunt ook niet zomaar middelen en zeggen dat de marge 15% is. Misschien is Amsterdam naar omzet gemeten wel veel kleiner dan Utrecht en draagt het dus minder bij aan de overall marge. Berekeningen in kubussen worden soms veel complexer omdat je altijd rekening moet houden met de multidimensionele wereld van de kubus.

Los van dit voorbeeld waarbij marges niet op te tellen zijn, moet ook rekening gehouden worden met de grote hoeveelheid cellen waar een berekening op van toepassing is. Stel dat de draaitabel van figuur 8.3 de hele kubus omvat en de gegevens toont op het laagst beschikbare detailniveau. In totaal bestaat de kubus dan uit 32 cellen (2 kwartalen * 4 plaatsen * 4 producten). Veelal worden totalen in een kubus vooraf berekend en opgeslagen en daar zijn ook cellen voor nodig. Daarmee komen er nog eens 18 cellen bij, wat het totaal aantal cellen brengt op 50. Dat houdt in dat er ook 50 keer een marge berekend moet worden. En voor elk van die berekeningen geldt dat de juiste aggregaties op elkaar gedeeld moet worden, anders kloppen de uitkomsten niet. Probeer je eens voor te stellen hoeveel berekeningen dat worden in een echte kubus met honderden producten, duizenden klanten en tientallen kenmerken voor zowel producten als klanten!

MDX en de OLAP query engine kennen deze problematiek. Als je de taal eenmaal kent, blijkt het alsnog redelijk eenvoudig een berekend veld zoals marge te maken. Sommige berekeningen blijken in MDX zelfs veel makkelijker te maken, of met een veel betere performance, dan in SQL. Op die manier kan een kubus dus ook functioneel nog meerwaarde bieden ten opzichte van een relationele datamart.

Een kubus is dus een soort draaitabel met een krachtige functietaal. Het eerste kenmerk sluit goed aan bij het gebruik van gegevens, het tweede verrijkt de gegevens ten opzichte van het datawarehouse. Dit zijn twee belangrijke argumenten voor het gebruik van kubussen. Performance van kubussen is een derde reden.

8.2.3 Aggregaties

In paragraaf 8.1 is een goede performance genoemd als eis die we stellen aan OLAP-databases. Er is een aantal redenen waarom kubussen veelal een betere performance bieden dan relationele databases. Dat zijn onder andere:

- een andere opslagstructuur;
- betere compressie;
- andere query engine;
- pre-aggregates.

Zelden worden bij analyses de gegevens geraadpleegd op het laagst beschikbare detailniveau. In het voorbeeld van figuur 8.3 is de grain van de onderliggende feitentabel waarschijnlijk een record voor elke transactie waar een klant fruit koopt. De getoonde getallen zijn dus de omzet van alle klanten van alle dagen binnen het kwartaal bij elkaar opgeteld. Dat kan om duizenden records gaan. De onderliggende dataset is dus groot en dat kan op zichzelf al leiden tot slechte performance. Bovendien moeten vanuit die datasets nog de getoonde totalen berekend worden.

In paragraaf 5.4.4 zijn aggregatietabellen besproken om het hierboven beschreven performanceprobleem op te lossen. Bij kubussen doe je over het algemeen iets vergelijkbaars. Aggregaties worden vooraf (meestal 's nachts) berekend zodat alleen de gevraagde getallen hoeven worden ingelezen als er een query wordt uitgevoerd. Er is dan geen sprake meer van een grote dataset (de onderliggende details worden ongemoeid gelaten) en er hoeft helemaal niets meer te worden berekend.

Een kubus gaat alleen veel slimmer met deze vooraf berekende aggregaties om dan een relationele database. Om dat uit te leggen moeten we eerst even kijken naar hiërarchieën.

Hiërarchieën

In veel analyses en rapportages wordt begonnen met het kijken naar gegevens op een hoog aggregatieniveau. Daarna kan worden ingezoomd naar gedetailleerdere levels. Neem bijvoorbeeld omzetten. Je begint te kijken naar jaaromzet om een jaar vervolgens uit te splitsen in vier kwartalen. Elk kwartaal kan weer verder worden uitgesplitst naar maanden en een maand kan weer worden opgesplitst in dagen. Je bent zo in een aantal stappen van hooggeaggregeerde informatie naar gedetailleerde informatie gegaan. De velden jaar, kwartaal, maand en dag vormen in deze volgorde een natuurlijke hiërarchie.

Er zijn veel voorbeelden te bedenken van natuurlijke hiërarchieën. Je kunt bijvoorbeeld omzet per land uitsplitsen naar omzet per provincie en verder naar per gemeente en per postcodegebiedje. Ook dit is een voorbeeld van een natuurlijke hiërarchie. Bij dit tweede voorbeeld moet je echter wel oppassen bij multinationals. Verschillende landen hebben een andere manier van het land verdelen in kleinere eenheden. De verschillende manieren van opsplitsen in verschillende landen hoeven niet overeen te komen. In Amerika hebben ze nog staten. Maar komt een staat in Amerika overeen met een provincie in Nederland?

Een natuurlijke hiërarchie is een drill-downpad waarvoor geldt dat er een een-op-veelrelatie bestaat van het hogere niveau naar lagere niveaus. Een jaar bestaat altijd uit meer kwartalen, maar één kwartaal is maar onderdeel van één specifiek jaar. Kwartaal 1 van 2017 bestaat uit drie verschillende maanden, maar januari 2017 is onderdeel van slechts één kwartaal, kwartaal 1 van 2017.

Een **natuurlijke hiërarchie** is een drill-downpad waarvoor geldt dat er een een-op-veelrelatie bestaat van het hogere niveau naar lagere niveaus.

In analyses is het ook gebruikelijk om niet-natuurlijke drill-downpaden te volgen. Je kunt bijvoorbeeld voor een kledingwinkel de omzet uitsplitsen per maat. Je zou vervolgens de omzet verder kunnen uitsplitsen naar kleur. Misschien is het zo dat kinderen voornamelijk felle kleuren kiezen terwijl volwassenen eerder kiezen voor neutrale kleuren. Als dat zo is, dan moet de afdeling inkoop dat natuurlijk wel weten. Dat maakt bovenstaande uitsplitsing dus zinvol. Tussen maat en kleur zit echter geen verband. Alle maten kunnen voorkomen in alle kleuren en alle kleuren kunnen voorkomen bij alle maten. Het drill-downpad van maat naar kleur wordt een niet-natuurlijke hiërarchie genoemd.

Voorbeeld

Kijk nog eens terug naar de draaitabel van figuur 8.3. Laten we voor het gemak er eens van uitgaan dat er per maand per stad duizend transacties plaatsvinden en dat elke transactie een record is in de database. Om de draaitabel van figuur 8.3 te maken moeten dan 6 maanden * 4 steden * 1000 records = 24.000 records gelezen worden. Daarna moeten deze records verwerkt worden door alle individuele omzetten op te tellen om tot de kwartaaltotalen per stad per fruitsoort te komen.

Stel nu eens dat de omzet per maand, per stad, per soort fruit al eerder berekend is. Dat betekent dat elke cel in de draaitabel berekend kan worden door slechts drie vooraf berekende maandtotalen uit de database te halen. Van 24.000 op te halen records gaat de query dan terug naar 32 cellen * 3 maanden = 96 records. Per cel hoeven nu nog maar drie getallen bij elkaar opgeteld te worden. Dit levert een enorme performancewinst op.

In deze berekening gaan we uit van de voorkennis dat de kalender een natuurlijke hiërarchie bevat. Ondanks dat het rapport kwartaalgegevens toont, kunnen we handig gebruikmaken van vooraf berekende aggregaties op een ander niveau, namelijk maand. Dat kan echter alleen met de kennis van hoe onze kalender in elkaar zit. Als je niet weet welke maanden samen een kwartaal vormen, valt dit betoog uiteen.

Ook als er geen natuurlijke hiërarchie bestaat, gaat de vlieger niet op. Stel dat je niet de maandtotalen maar weektotalen vooraf had berekend. Hoeveel winst had je dan behaald? Helaas blijkt dat weektotalen zinloos zijn voor dit rapport. Een kwartaal kan halverwege een week beginnen en/of eindigen en dus kun je vanuit weektotalen nooit met zekerheid zeggen wat het kwartaalresultaat is.

Waarom dit hele verhaal over aggregaties en hiërarchieën? Je kunt bij het ontwerpen van een kubus opgeven dat de gegevens natuurlijke hiërarchieën bevatten. Dat is mooi voor het model, want de gebruiker kan nu gemakkelijk in rapportages en analyses gebruikmaken van deze hiërarchieën door drill-downrapporten te maken. Nog mooier is dat de query engine van een kubus in staat is deze hiërarchieën te gebruiken zoals hierboven beschreven. Automatisch zullen bij het verwerken van query s vooraf berekende aggregaties gebruikt worden waar dat mogelijk is. Het grote verschil met SQL is dat je bij een relationele database die kennis zelf moet hebben en moet verwerken in de query. Je moet de aggregatietabel gebruiken in plaats van de detailtabel. Bij MDX blijft de query precies hetzelfde. De kennis zit nu in de query engine, niet in het hoofd van de query-programmeur.

8.2.4 Model of database?

Een kubus is een OLAP-database, ofwel een multidimensionele database. Microsoft noemt hun SQL Server Analysis Services-kubus tegenwoordig ook wel de UDM, Unified Dimensional Model. We noemen dat hier vanwege het woord model. Is het nu een model of een database of misschien wel beide?

Een kubus is in alle gevallen een model zoals bedoeld en beschreven in paragraaf 8.1. Bij het ontwerpen van de kubus moet de ontwerper altijd de eindgebruiker in gedachte hebben. Op basis van de informatieanalyse weet je welke rapporten er gewenst zijn in de organisatie, hoe kolommen moeten gaan heten en welke grootheden tegen elkaar uitgezet moeten worden. Dat alles komt samen in het model dat de kubus is. De interface van de kubus, het model, is volledig in de taal (het jargon) van de gebruiker. Er zijn geen technische naamgevingsconventies meer, geen technische of auditing-kolommen, geen inner en outer joins. Het model is een abstractielaag voor al die technische zaken.

Is een kubus naast een model ook een database? Eigenlijk is dat een keuze. In de praktijk wordt er onderscheid gemaakt tussen MOLAP en ROLAP en ook de varianten HOLAP en DOLAP komen voor.

MOLAP

MOLAP staat voor 'multi dimensional OLAP' en wordt vaak gezien als hetzelfde als OLAP. Bij MOLAP worden alle gegevens in de kubus zelf opgeslagen. Het is dus een echte database. De gegevens zijn opgeslagen in een relationeel datawarehouse én

redundant ook nog eens in de kubus. Dat kost uiteraard opslagcapaciteit. Bovendien moeten de gegevens eens in de zoveel tijd (veelal elke nacht) vanuit het datawarehouse opnieuw in de kubus geladen worden. De informatie die je uit de kubus krijgt, loopt dus altijd een beetje achter. Het voordeel van MOLAP, de meest gebruikte vorm bij kubussen, is de goede performance die het levert.

ROLAP

ROLAP staat voor relational OLAP. De gegevens zitten in een onderliggende relationele database (waarschijnlijk de datamart). De kubus is nu niets anders dan een semantisch model. Alle query's die worden afgevoerd op de kubus, worden on-the-fly vertaald naar SQL statements die worden uitgevoerd op de datamart. Er is geen sprake van gegevensredundantie maar dat gaat ten koste van de performance. ROLAP heeft een slechtere performance dan MOLAP.

Sommige mensen zeggen dat ROLAP staat voor realtime OLAP. Een nadeel van MOLAP is dat de kubus op basis van een schema ververst moet worden. Het duurt daarmee enige tijd (vaak een dag) totdat wijzigingen ook zichtbaar worden in de kubus. Als de kubus bij alle gebruik zijn gegevens uit een onderliggende database haalt, worden wijzigingen in die onderliggende database dus meteen zichtbaar in de kubus; realtime.

HOLAP en DOLAP

HOLAP staat voor Hybrid OLAP, een mengvorm van MOLAP en ROLAP. Vooraf berekende aggregaties worden opgeslagen in de kubus (MOLAP) maar de detailgegevens blijven achter in de datamart (ROLAP). DOLAP staat voor Desktop OLAP en representeert technieken om de informatie offline beschikbaar te maken zodat bijvoorbeeld thuis met de laptop ook gewerkt kan worden.

8.2.5 Tot slot

Samengevat heeft een kubus drie voordelen te bieden:

- Een kubus is een semantisch model.
- Een kubus heeft een krachtige rekentaal: MDX.
- Een kubus heeft goede performance.

De interface van een kubus vormt een semantisch model van de bedrijfsprocessen. Met de juiste front-endtools kunnen gebruikers die geen technische kennis hebben maar wel verstand van de processen, goed, gemakkelijk en snel analyses doen op hun gegevens.

Ontwikkelaars kunnen door de kracht van de OLAP query engine en de taal MDX meerwaarde toevoegen aan de gegevens ten opzichte van diezelfde gegevens in een relationele database. MDX heeft meer relevante statistische mogelijkheden dan SQL.

Een kubus kan een betere performance bieden dan de meeste relationele databases. Het doel is willekeurige analysis op de gegevens te doen (*to slice and dice*) en direct antwoorden krijgen (*answers at the speed of thought*). Of de praktijk zo mooi is, is de vraag, maar een kubus komt hiermee een stuk verder dan een relationele database.

8.3 Voorbeeld in-memorymodel

Kubussen bieden veel functionaliteit en worden veel gebruikt in huidige Business Intelligence-omgevingen. In de praktijk blijkt dat het vaak moeilijk is goede kubussen te bouwen. Daardoor worden BI-projecten duur en komt de gewenste informatie misschien niet op het goede moment. Bovendien werkt de kubus met MDX als vraagtaal, wat met regelmaat een te hoge drempel opwerpt.

Met de toegenomen geheugencapaciteit van servers en het toegenomen aantal cores in servers, komen er steeds meer producten die dezelfde soort mogelijkheden bieden als kubussen, maar dan met eenvoudiger techniek. De gewenste performance wordt gehaald door alles in-memory, dus in het werkgeheugen van de server te doen. Er wordt afgestapt van het idee dat een database bestaat uit bestanden op harde schijven waar gegevens inzitten. De hele database, dus de volledige dataset, zit in het geheugen.

Net zoals in de vorige hoofdstukken gaan we een en ander concreet maken door een voorbeeld uit te werken. Ditmaal niet met SQL Server maar met Excel 2016. Tegelijkertijd maken we nog wel gebruik van de SQL Server-database AdventureWorksDW2014 die we ook in andere hoofdstukken hebben gebruikt. Lees de rest van de paragraaf op *www.leerboekbusinessintelligence.nl*.

8.4 Tot slot

Een voordeel van Excel in combinatie met een model zoals je dat in de vorige paragraaf hebt gezien, is de analytische kracht van de combinatie. In plaats van dat de gebruiker een vrij statische lijst met gegevens krijgt, is er een model dat willekeurig naar eigen inzicht en ad hoc gebruikt kan worden. Een gebruiker kan de winst per product bekijken. Hij kan deze winst uitsplitsen naar de kleur van het betreffende product en in een vervolgstap kijken of de maand van invloed is op de kleur. Het maakt niet uit of deze vraag (is maand van invloed op welke kleur producten verkocht worden?) in de informatieanalyse zo letterlijk naar voren is gekomen.

Hiermee is de stap gemaakt van rapporteren naar analyseren. Rapporteren is vrij statisch. De aandeelhouders willen maandelijks inzicht in kosten en opbrengsten. Daarvoor zijn grootboekposten afgesproken en in een rapport dat elke maand hetzelfde is (behalve de inhoud natuurlijk) wordt de stand van zaken gerapporteerd. Omdat dit elke keer hetzelfde is, kan het tijdens een informatieanalyse goed beschreven worden. Alle eisen zijn duidelijk en de structuur en inhoud zijn duidelijk.

Er is echter veel meer te doen met alle informatie die een bedrijf tot zijn beschikking heeft. Sales en marketing zijn geïnteresseerd in wie welke producten koopt. Wat koopt een klant nog meer? Zijn er cross-sellingmogelijkheden? Zijn er trends te ontdekken die ons in staat stellen eerder en beter in te spelen op de vraag vanuit de markt? Trends die in traditionele rapportages pas achteraf, en dus te laat, zichtbaar worden? Bij dit soort vragen is het doel van de vragen duidelijk, maar op voorhand zijn de gestelde vragen zelf niet duidelijk. De data-analist moet met de gegevens kunnen spelen. Dat is wat kubussen en in-memorymodellen brengen.

In de hoofdstukken 3 tot en moet 6 lag de nadruk op het verzamelen van de juiste gegevens om die vervolgens met de juiste kwaliteit in de juiste structuur op te slaan.

Hoofdstuk 7 nuanceerde de behoefte aan structuur en focuste met analytics al op het meer praktische gebruik. In dit hoofdstuk lag de focus echt op het gebruik van de gegevens. Hoe kijkt de gebruiker ertegenaan? Hoe moeten we de gegevens presenteren naar die gebruiker?

Maar het echte presenteren, de communicatie uit analytics, hebben we nog niet gedaan. In het volgende hoofdstuk ga je kijken naar het eindresultaat. Wat zien de gebruikers van Business Intelligence? Hoofdstuk 9 houdt zich bezig met het front-end.